INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PIAUÍ

CAMPUS TERESINA-CENTRAL

DIRETORIA DE ENSINO

Estrutura de Dados II –

Percurso em Extensão e Remoção em Árvore

- Aula 5 -

Professora: Elanne Cristina O. dos Santos

elannecristina.santos@gmail.com
elannecristina.santos@ifpi.edu.br

Percurso em Extensão

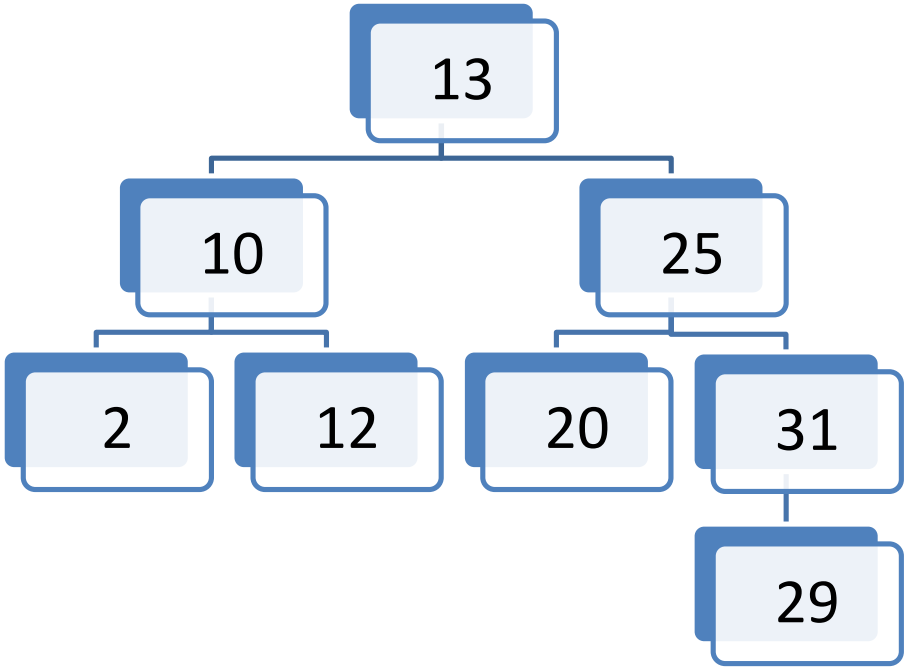
- Do nível mais baixo (ou mais alto) nível a nível, visitando nós em cada nível da esquerda para a direita (ou da direita para a esquerda).

- Implementação direta quando uma fila é usada. Depois que um nó é visitado, seus filhos são colocados no final da fila.

- Todos os nós no nível N precisam ser visitados antes de visitar quaisquer nós no nível n+1.

Ordens de Percurso

- Na árvore abaixo:



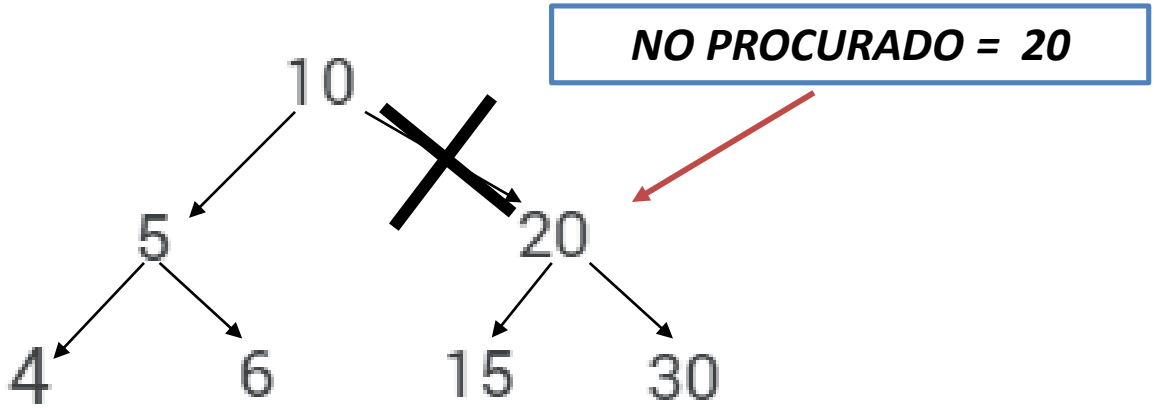
- o percurso em extensão será 13,10,25,2, 12, 20, 31 e 29.

Percurso em extensão do topo para baixo da esquerda pra direita (similar em fig. 6.10 – pg 196)

```
void percusoExtensao(){
    queue<ArvoreNo<T>*> f;
    ArvoreNo<T> *p=root;
    if (p!=NULL){
        f.push(p);
        while (!f.empty()){
            p=f.front();
            visit(p);
            f.pop();
            if (p->left!=0)
                f.push(p->left);
            if (p->right!=0)
                f.push(p->right);
        }   }
}
```

Remoção

- Uma primeira opção seria apagar o nó procurado e também os filhos do nó , caso ele tenha filhos.
- Ex:



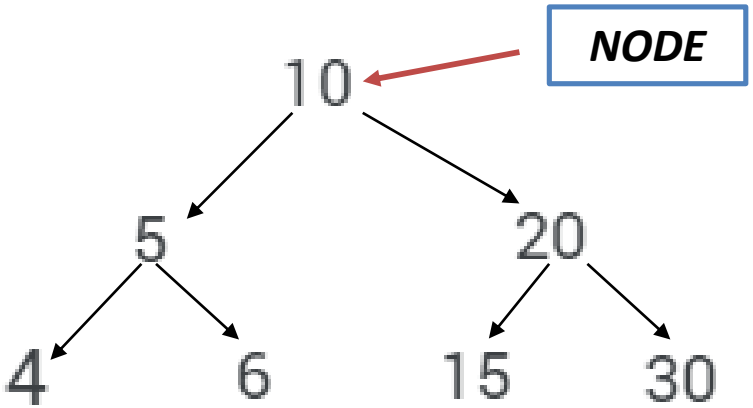
Para atividade desta semana:

- Implemente a remoção de um nó da árvore sendo que , se o mesmo tiver filhos, os filhos também devem ser apagados.

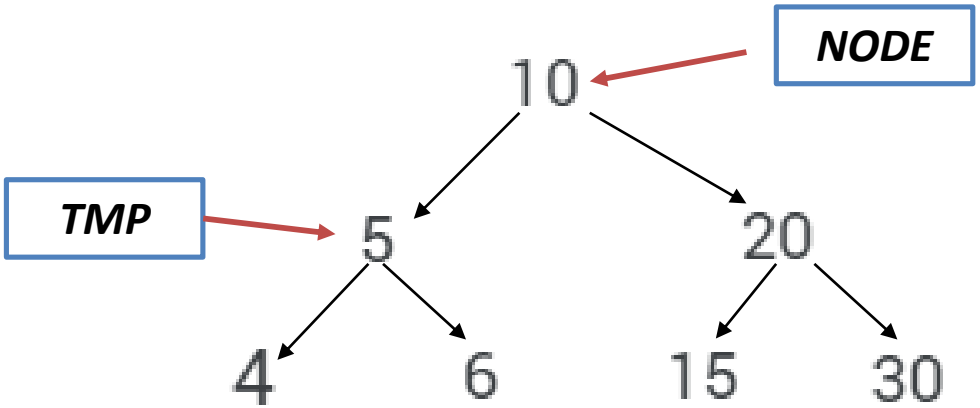
Remoção

- E quando o nó removido tem filhos e é preciso reorganizar a árvore de modo a não perder esses filhos?

Remoção por fusão

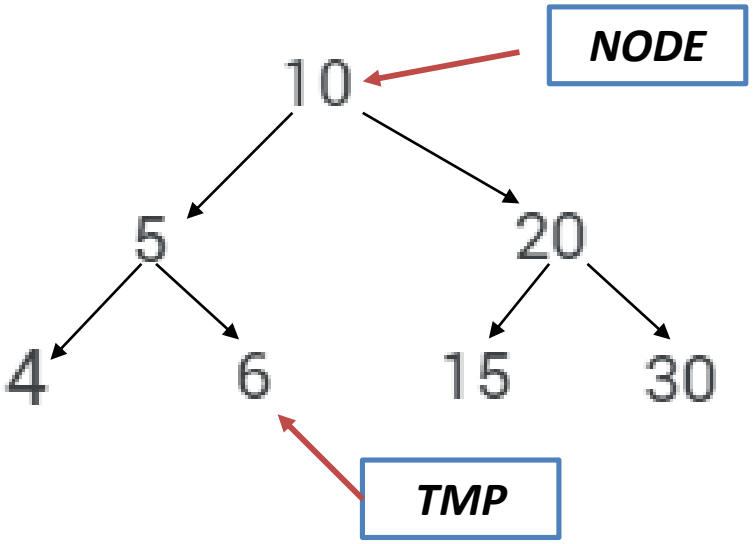


Remoção por fusão



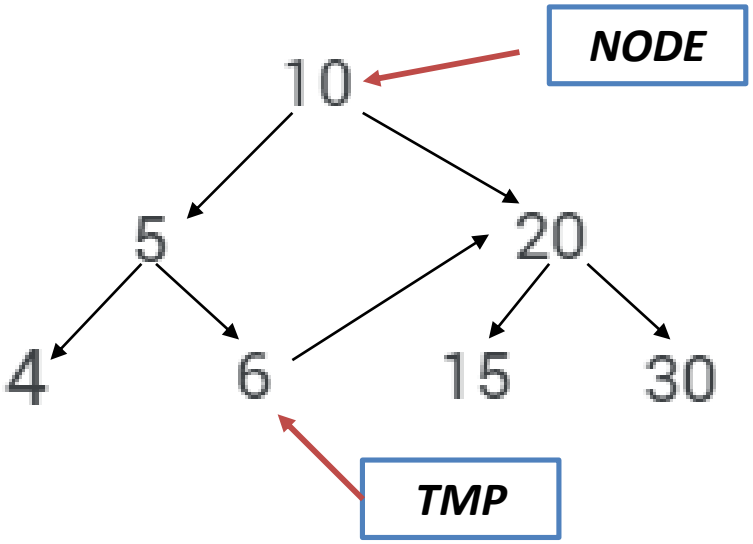
1. MOVA-SE PARA A ESQUERDA

Remoção por fusão



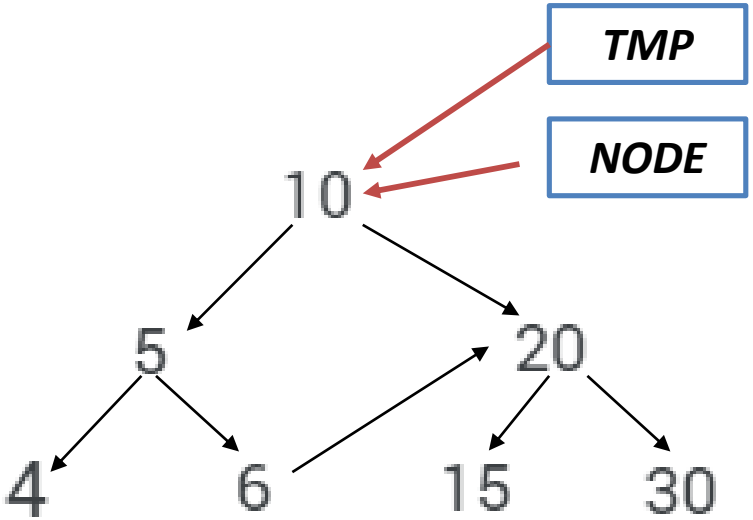
2. E ENTÃO MOVA-SE PARA A DIREITA TANTO QUANTO POSSÍVEL

Remoção por fusão



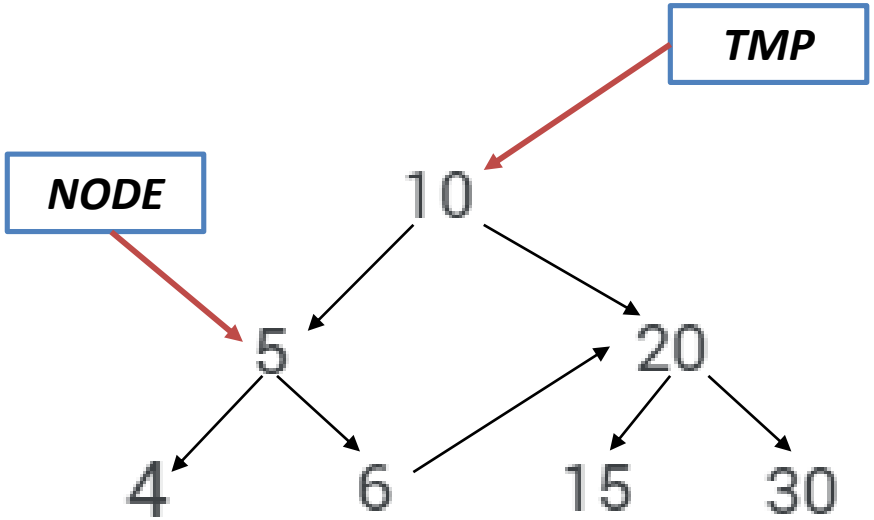
3. ESTABELEÇA O VÍNCULO ENTRE O NÓ MAIS A DIREITA DA SUBÁRVORE ESQUERDA E DA SUBÁRVORE DIREITA

Remoção por fusão



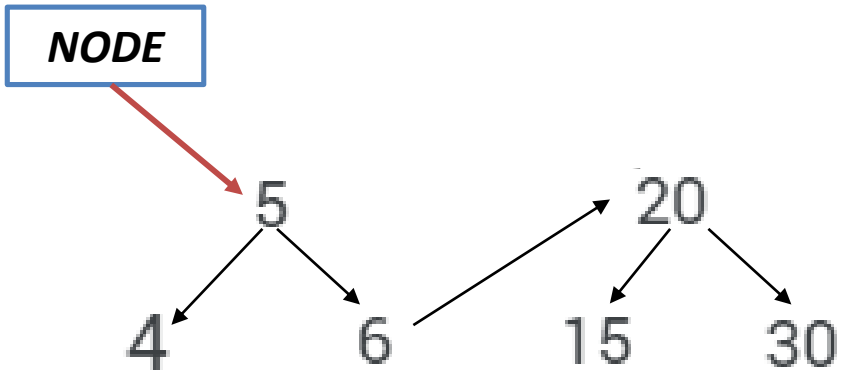
4. ATUALIZE TMP.
TMP APONTA PARA NODE

Remoção por fusão



5. ATUALIZE NODE.
NODE APONTA PARA NODE->LEFT

Remoção por fusão

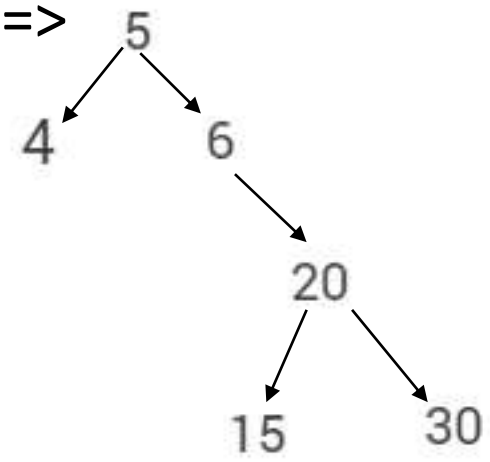


6. DELETE TMP.

Remoção por fusão

- Pode resultar no **aumento da altura da árvore**.

No exemplo anterior=>

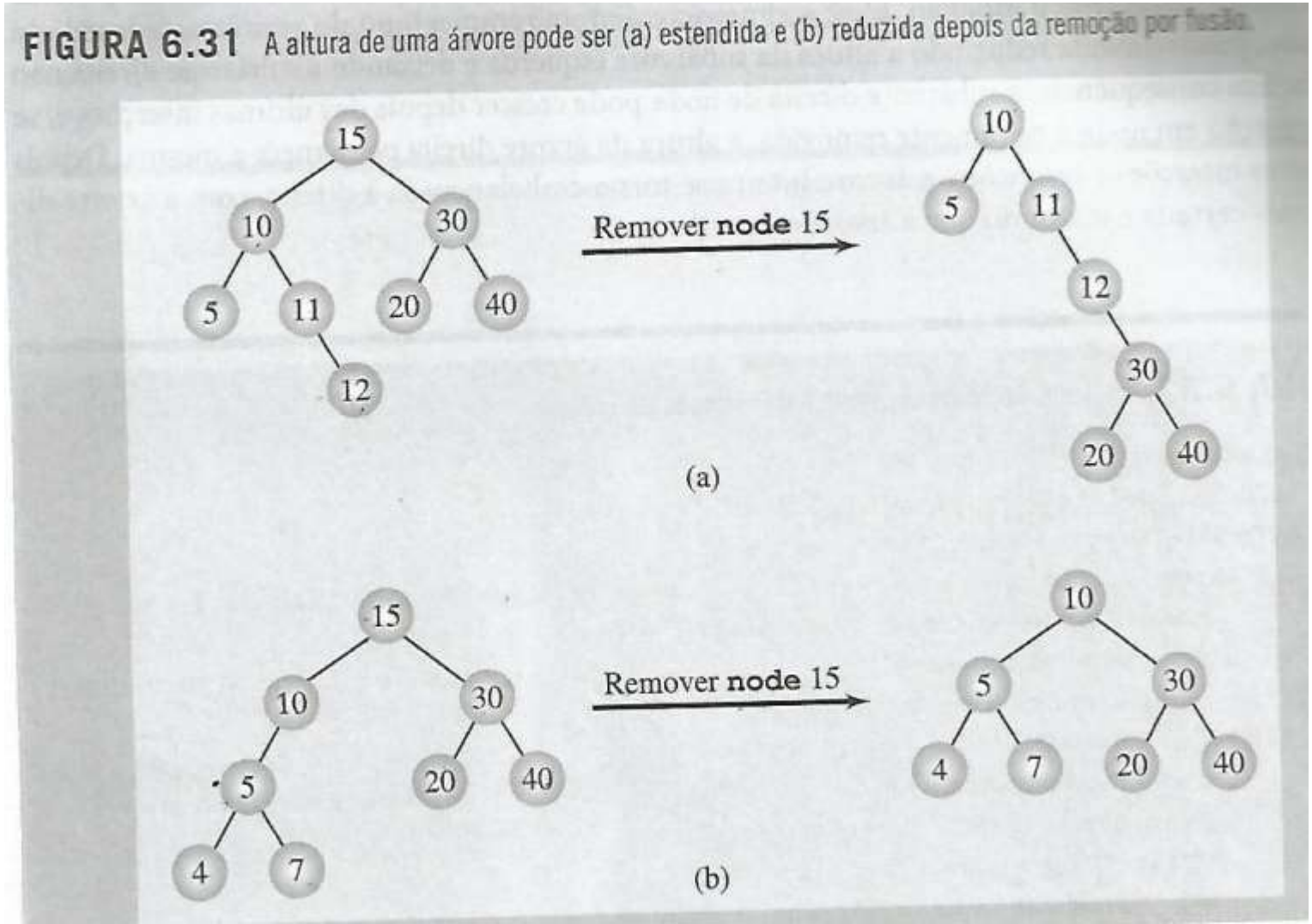


- Em alguns casos a nova árvore pode ser altamente desbalanceada.

- O algoritmo não é ineficiente, mas seria preferível um algoritmo que não aumentasse a altura da árvore.

Remoção por Fusão:

- Algumas vezes pode também acontecer da altura ser reduzida (ver pag. 215 do livro).



Pag. 213 – figura 6.29

```
template<class T>
void BST<T>::deleteByMerging(BSTNode<T>*& node) {
    BSTNode<T> *tmp = node;
    if (node != 0) {
        if (!node->right)              // o no nao tem filho a direita; seu filho
            node = node->left;         // a esquerda (se houver) e anexado a
                                       // seu ascendente;
        else if (node->left == 0)      // o no nao tem filho a esquerda; seu filho
            node = node->right;        // a direita e anexado a seu ascendente;
        else {                         // esteja preparado para fundir as subarvores;
            tmp = node->left;          // 1. mova-se para a esquerda
            while (tmp->right != 0)    // 2. e entao para a direita tanto quanto
                                       // possivel;
                tmp = tmp->right;
            tmp->right =               // 3. estabeleca o vinculo entre
                node->right;           // o no mais a direita da subarvore
                                       //esquerda e da subarvore direita;
            tmp = node;                // 4.
            node = node->left;         // 5.
        }
        delete tmp;                    // 6.
    }
}
```

Implementação

```
void findAndDeleteByMerging(T el){
    ArvoreNo *node=root;
    ArvoreNo *prev=0;
    while (node!=0){
        if (node->el == el)
            break;
        prev=node;
        if (el < node->el)
            node = node->left;
        else node= node->right;
    }
    if (node!=0 && node->el==el)
        if (node==root)
            remova(root);
        else
        if (prev->left==node)
            remova(prev->left);
        else remova(prev->right);
        else if (root!=0)
            cout<<"elemento "<<el<<"nao esta na árvore!\n";
        else cout<<"a arvore esta vazia\n";
}
```

Pag. 214 – figura 6.29